5. Voice Control for Multi-Point Navigation

5.1. Function Description

Using the voice recognition module, you can achieve multi-point navigation in an established map through voice control.

Note: Before using the functions in this section, please first learn to use the [Lidar Course ----- Mapping and Navigation Function] module.

5.2. Preparation

This section requires the use of a voice control device. Before running this lesson, you need to perform the following preparatory operations:

5.2.1. Pre-use Configuration

Description: Since the Muto series robots are equipped with various lidar devices, the factory system has been configured with routines for multiple devices. However, because the product cannot be automatically identified, you need to manually set the lidar model.

After entering the container: make the following modifications according to the lidar type:

```
root@ubuntu:/# cd
root@ubuntu:~# vim .bashrc
```

```
# env
alias python=python3
export ROS_DOMAIN_ID=26

export ROBOT_TYPE=Muto
export RPLIDAR_TYPE=a1          # a1, 4ROS
echo "-----------------------------------"
echo -e "ROS_DOMAIN_ID: \033[32m$ROS_DOMAIN_ID\033[0m"
echo -e "my_robot_type: \033[32m$ROBOT_TYPE\033[0m | my_lidar: \033[32m$RPLIDAR_TYPE\033[0m"
echo "-----------------------------------"

#colcon_cd
source /usr/share/colcon_cd/function/colcon_cd.sh
export _colcon_cd_root=/root/yahboomcar_ros2_ws/yahboomcar_ws
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash

#ros2
source /opt/ros/foxy/setup.bash
source /root/yahboomcar_ros2_ws/yahboomcar_ws/install/setup.bash
source /root/yahboomcar_ros2_ws/software/library_ws/install/setup.bash
```

After the modification is complete, save and exit vim, then execute:

```
root@jetson-desktop:~# source .bashrc
-----------------------------------
ROS_DOMAIN_ID: 26
my_robot_type: Muto | my_lidar: a1
-----------------------------------
root@jetson-desktop:~#
```

You can see the currently modified lidar type is my_lidar: a1.

5.3. Configure Navigation Points

1. Enter the container, refer to [Docker Course ----- 5. Enter the Muto's docker container], and execute the following command in a separate terminal:

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

2. Open the virtual machine, configure multi-machine communication, and then execute the following command to display the rviz node:

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

3. Start the lidar odometry in the docker container:

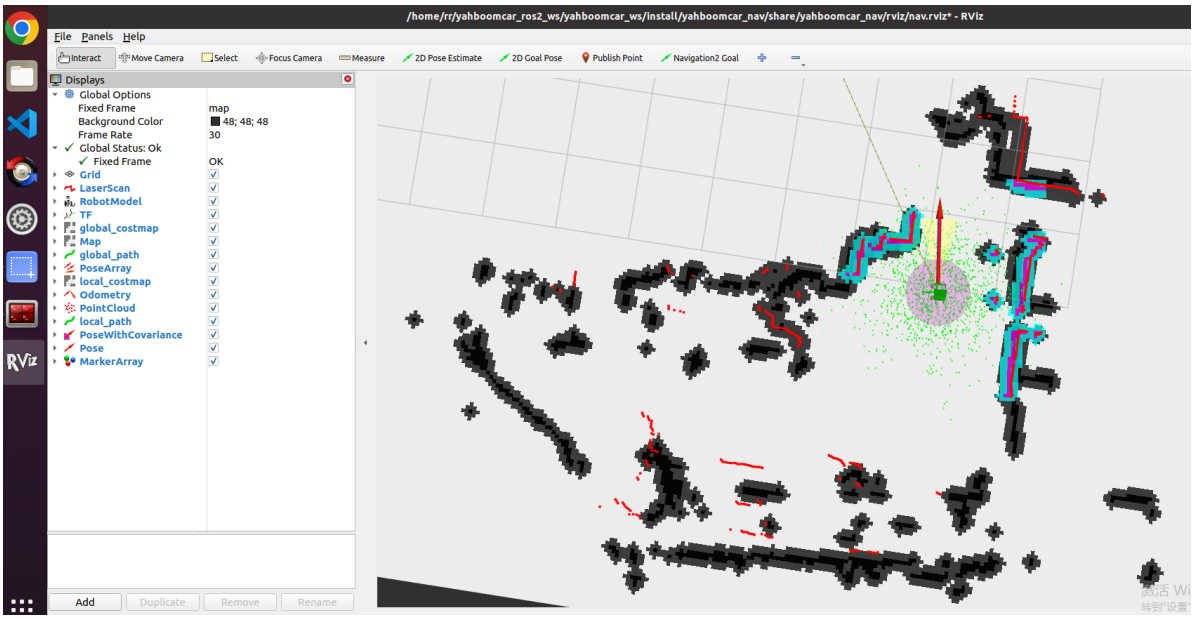
```
ros2 launch rf2o_laser_odometry rf2o_laser_odometry.launch.py
```

4. Execute the navigation node in the docker container:

```
ros2 launch hexapod_nav hexapod_navigation_mppi.launch.py autostart:=true
map:=/root/yahboomcar_ros2_ws/maps/test_map.yaml
# The map path above is the map path you want to load during navigation.
test_map.yaml is the map name, which means that the test_map will be loaded
during navigation. If it is another name, it needs to be changed accordingly
here.
```

5. At this time, click [2D Pose Estimate] in the rviz interface of the virtual machine, and then compare the pose of the car to mark an initial pose for the car on the map;

   The display after marking is as follows:



6. Compare the coincidence of the radar scanning points and obstacles. You can set the initial pose of the car multiple times until the radar scanning points and obstacles roughly coincide;

7. Open another terminal to enter the docker container and execute:

```
ros2 topic echo /goal_pose   # Listen to the /goal_pose topic
```

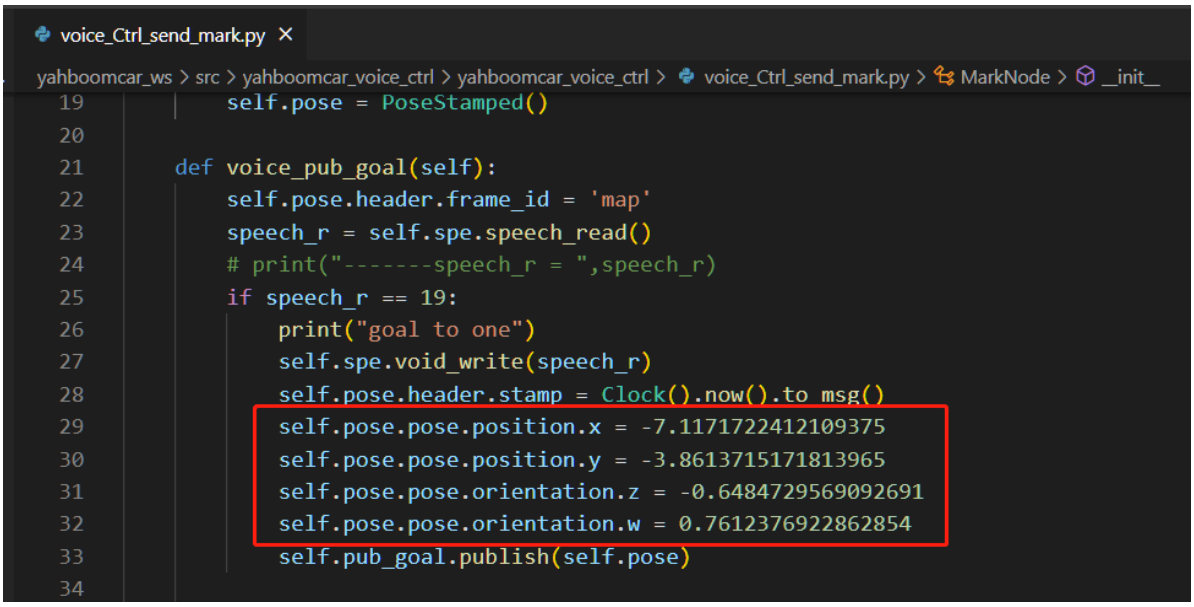
8. Click [2D Goal Pose] to set the first navigation target point. At this time, the car starts to navigate, and the monitored topic data will be received in step 6:

```
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 topic echo /goal_pose
header:
  stamp:
    sec: 1682416565
    nanosec: 174762965
  frame_id: map
pose:
  position:
    x: -7.258232593536377
    y: -2.095078229904175
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.3184907749129588
    w: 0.9479259603446585
```

9. Open the code at the following location:

```
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl/yahboomcar_voic
e_ctrl/voice_Ctrl_send_mark.py
```

Modify the pose of the first navigation point to the one printed in step 7:



10. Modify the poses of the other 4 navigation points in the same way.

5.4. Use Voice for Multi-Point Navigation

1. Enter the container, refer to [5. Enter the Muto's docker container], and execute the following command in a separate terminal:

```
ros2 launch yahboomcar_nav laser_odom_bringup_launch.py
```

2. Open the virtual machine, configure multi-machine communication, and then execute the following command to display the rviz node:

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

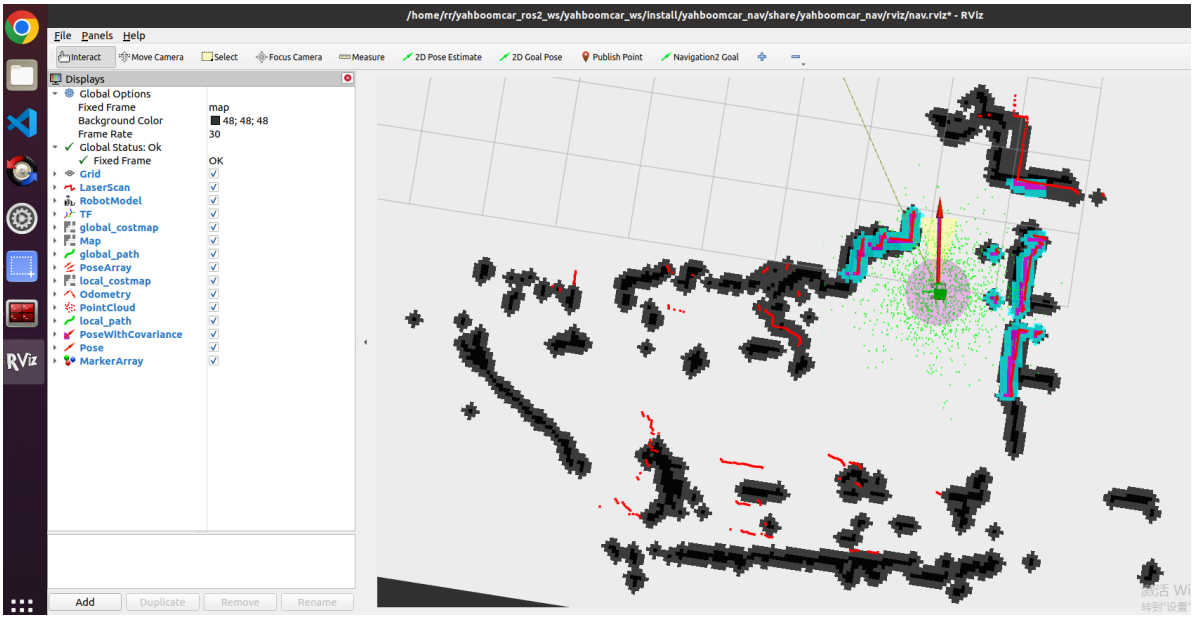
3. Execute the navigation node in the docker container:

```
ros2 launch yahboomcar_nav navigation_teb_launch.py

# The above method loads the yahboomcar map by default. If you need to load other
maps, please use the following method to start:
ros2 launch yahboomcar_nav navigation_teb_launch.py
map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/test.yaml
# The map path above is the map path you want to load during navigation.
test.yaml is the map name, which means that the test map will be loaded during
navigation. If it is another name, it needs to be changed accordingly here.
```

4. At this time, click [2D Pose Estimate] in the rviz interface of the virtual machine, and then compare the pose of the car to mark an initial pose for the car on the map;

The display after marking is as follows:



6. Compare the coincidence of the radar scanning points and obstacles. You can set the initial pose of the car multiple times until the radar scanning points and obstacles roughly coincide;

7. Open two more terminals to enter the docker container, and execute the commands to start the voice control navigation node and the voice recognition node respectively:

```
ros2 launch yahboom_speech speech.launch.py
```

```
ros2 run yahboomcar_voice_ctrl voice_Ctrl_send_mark
```

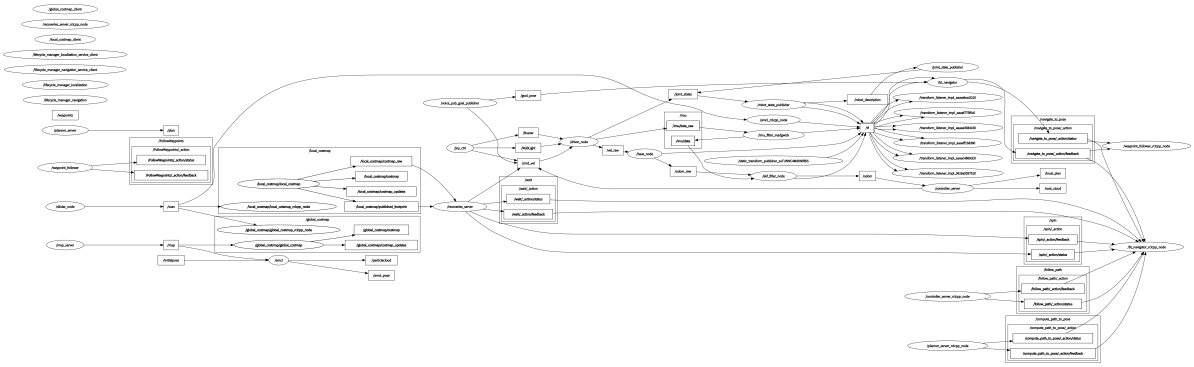
8. Say "Hello, yahboom" to the voice module on the car to wake it up. After hearing the voice module's feedback broadcast "I'm here", continue to say "go to point A"; the voice module will feedback broadcast "OK, I am going to the point A", and the car will start navigating to position one. The navigation of other positions can be used in the same way. Refer to the table below for voice control function words:

Function Word	Voice Recognition Module Result	Voice Broadcast Content
GO-TO-THE-POINT-A	19	ok I am going to the point a
GO-TO-THE-POINT-B	20	ok I am going to the point b
GO-TO-THE-POINT-C	21	ok I am going to the point c
GO-TO-THE-POINT-D	32	ok I am going to the point d
BACK-TO-ORIGIANL	33	ok i am return back

5.5. Node Analysis

5.5.1. Display Computation Graph

```
rqt_graph
```



5.5.2. Voice Control Node Details

```
rr@rr-pc:~$ ros2 node info /voice_pub_goal_publisher
/voice_pub_goal_publisher
  Subscribers:

  Publishers:
    /cmd_vel: geometry_msgs/msg/Twist
    /goal_pose: geometry_msgs/msg/PoseStamped
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
  Service Servers:
    /voice_pub_goal_publisher/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /voice_pub_goal_publisher/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /voice_pub_goal_publisher/get_parameters: rcl_interfaces/srv/GetParameters
    /voice_pub_goal_publisher/list_parameters: rcl_interfaces/srv/ListParameters
    /voice_pub_goal_publisher/set_parameters: rcl_interfaces/srv/SetParameters
    /voice_pub_goal_publisher/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
```